Lecture 09: TCP Client-Server Programming I

Objectives:

- Learn about the **IPAddress** and **IPEndPoint** classes
- Learn how to write a TCP server using the **TcpListener** class
- Learn how to use ***Telnet*** to test a server application
- Learn how to write a TCP client using the **TcpClient** class

1. IPAddress and IPEndPoint classes

These are two classes required in creating both server and client objects.

IPAddress class:

This class is used to represent an IP address as an object.

The two constructors of the **IPAddress** class are as follows:

```
public IPAddress(long address)
public IPAddress(byte[] address)
```

These constructors are rarely used, as we hardly have the IP address represented in bytes or long format.

Instead of using these constructors, the static method, ***Parse()*** of the **IPAddress** class**,** discussed below, is often used to create an instance of **IPAddress**.

Following are two important methods of the IPAddress class are:

static IPAddress Parse(String address)	Takes an IP address as a string and returns an IPAddress object
static bool IsLoopback(IPAddress address)	returns true if *address* is a loop back address

IPAddress class also has a number of public **read-only** properties as described below:

Any	Represents any IP address available on the local system.
Broadcast	Represents broadcast address.

Loopback	Represents loopback address.

IPEndPoint:

TCP/UDP client-server applications usually connect or listen to a pair of **IPAddress** and a **por**t number. The **IPEndPoint** class allows these two components to be represented as a single object.

The must frequently used constructor of this class has the form:

```
public IPEndPoint(IPAddress address, int port)
```

It also has properties, **IPAddress** and **Port**, that can be used to get the corresponding components of the end-point,

2. Creating a TCP server using the *TcpListener* class

The easiest way to create a TCP server is by using the TcpListener class.

This class has the following constructors:

TcpListener(IPAddress, int port)	Binds the server to a specific IPAddress object and port number.
TcpListener(IPEndPoint ie)	Binds the server to an IPEndPoint object.

Some of the important methods of the TcpListener class are:

`void Start()`	Starts the server
`TcpClient AcceptClient()`	Accepts connection from a TcpClient
`void Stop()`	Stops the server
`bool Pending()`	Determines if there are pending connections

After a TcpListener object is created, it must be started by calling the ***Start()*** method.

The ***AcceptTcpClient()*** method is then called to establish a connection with a client. Note that this method will block until a client is received. After establishing a connection, this method returns an instance of **TcpClient**. The TcpClient instance has a method, ***GetStream()***, which returns a **NetworkStream** object. It is this NetworkStream object that is used both for reading and writing data from and to the client.

NetworkStream behaves exactly like the **FileStream** we saw earlier. It can be used to read/write byte array. Also it can be used to create **StreamReader**/**StreamWriter** object for exchanging text messages with the client.

Example 1:

The following example creates a simple Tcp server. It simply echo back the string message it receives from the client.

```
using System;
using System.Net;
using System.Net.Sockets;
using System.IO;

class SimpleTcpServer {
    public static void Main() {
        TcpListener server = new TcpListener(IPAddress.Any,
9050);
        server.Start();

        Console.WriteLine("Waiting for Client...");
        TcpClient client = server.AcceptTcpClient();
        Console.WriteLine("Connected with a client");

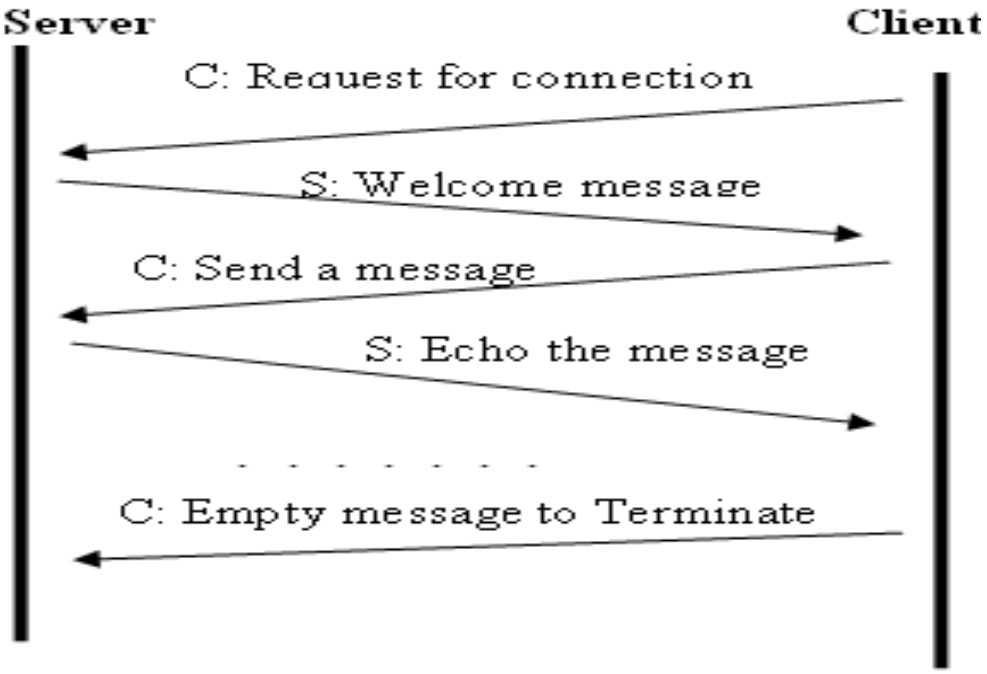
        NetworkStream stream = client.GetStream();
        StreamReader reader = new StreamReader(stream);
        StreamWriter writer = new StreamWriter(stream);

        writer.WriteLine("Welcome to my test server");
        writer.Flush();
        String line = null;
        while((line = reader.ReadLine()).Length != 0) {
            Console.WriteLine(line);
            writer.WriteLine(line);
            writer.Flush();
        }
        client.Close();
```

```
            server.Stop();
        }
}
```

One important thing that must be decided when writing a network program is the **protocol** - the rule that governs communication between the server and the client.

The protocol used in the above client-server example can be summarized as follows:



3. Using Telnet to test a server application

We can easily test our server above using the Microsoft Telnet program that comes standard with all Windows platforms.

To start the Telnet, simply go command window and type:

C:\>telnet ipaddress port

where ipaddress and port are the IP address and the port number on which the server is running.

4. Creating a TCP client using the *TcpClient* class

While a **TcpListener** object must be bound to a **local IP** and port number, a **TcpClient** instance must be bound to a **remote IP** and port number. The IP and port number on which a Tcp server is listening to.

The following are the constructors of the TcpClient class:

`TcpClient(string host, int port)`
`TcpClient(IPEndPoint localEP)`
`TcpClient()`

The first constructor is by far the most frequently used. You just need to give the **host name** or **ip address** followed by the **port number** of the server.

The second constructor is suitable for machines having more than one interface card. Both this and the last constructor need to be connected to a remote server using the **Connect** method discuss below.

The most important methods of TcpClient class are shown below:

`void Connect(string host,  int port)`
`NetworkStream GetStream()`
`Close()`

After a client is done, the Close method is called to disengage from the server.

The following example is a Tcp client application designed to work with the Tcp server discussed above:

```
using System;
using System.Net;
using System.Net.Sockets;
using System.IO;
```

```
class SimpleTcpClient {
    public static void Main() {
        TcpClient client = new TcpClient("localhost", 9050);

        NetworkStream stream = client.GetStream();
        StreamReader reader = new StreamReader(stream);
        StreamWriter writer = new StreamWriter(stream);

        String input = reader.ReadLine();
        Console.WriteLine(input);

        String line = null;
        do {
            Console.Write("Enter Message for Server <Enter
to Stop>: ");
            line = Console.ReadLine();
            writer.WriteLine(line);
            writer.Flush();
            if (line.Length != 0) {
                line = "Echo: "+ reader.ReadLine();
                Console.WriteLine(line);
            }
        } while(line.Length != 0);
        client.Close();
    }
}
```